Quickstarts

Bridge USDC to Arc

Copy page

Learn how to bridge USDC to Arc via CCTP with Bridge Kit

> **Cross-Chain Transfer Protocol (CCTP)** is a permissionless onchain utility that facilitates USDC transfers securely between supported blockchains via native burning and minting. For more info, visit the **CCTP** docs.

This tutorial walks you through bridging USDC to Arc Testnet with **Bridge Kit**. Select the tab that matches your adapter setup.

Circle Wallets | **Solana** | **Viem**

Prerequisites

Before you begin, make sure you have:

1. Installed **Node.js v22+**
2. A **Circle Developer Console** account
3. An API key created in the Console:
   Keys → Create a key → API key → Standard Key
4. Your **Entity Secret registered**

Step 1. Set up your project

In this step, you prepare your project and environment.

1.1. Create the project and install dependencies

Create a new project directory and install Bridge Kit with the Circle Wallets adapter:

Shell

```
mkdir crosschain-transfer && cd crosschain-transfer
npm init -y
npm pkg set type=module
npm pkg set scripts.start="tsx --env-file=.env index.ts"

npm install @circle-fin/bridge-kit @circle-fin/adapter-circle-wallets
npm install --save-dev tsx typescript @types/node
```

1.2. Configure TypeScript (optional)

This step is optional. It helps prevent missing types in your IDE or editor.

Create a `tsconfig.json` file:

```
npx tsc --init
```

Then, update the `tsconfig.json` file:

```
cat <<'EOF' > tsconfig.json
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "types": ["node"]
  }
}
EOF
```

1.3. Set environment variables

Create a `.env` file in the project directory:

.env

```
CIRCLE_API_KEY=YOUR_API_KEY
CIRCLE_ENTITY_SECRET=YOUR_ENTITY_SECRET
EVM_WALLET_ADDRESS=YOUR_EVM_WALLET_ADDRESS
SOLANA_WALLET_ADDRESS=YOUR_SOLANA_WALLET_ADDRESS
```

- `CIRCLE_API_KEY` is your Circle Developer API key for the Circle Wallets adapter.
- `CIRCLE_ENTITY_SECRET` is your registered entity secret used to authorize developer-controlled wallet operations.
- `EVM_WALLET_ADDRESS` is your Arc Testnet or EVM developer-controlled wallet address.
- `SOLANA_WALLET_ADDRESS` is your Solana Devnet developer-controlled wallet address.

The `npm run start` command loads these variables from `.env` using Node.js native env-file support.

> Prefer editing `.env` files in your IDE or editor so credentials are not leaked to your shell history.

Step 2. Set up your wallets

In this step, you create developer-controlled wallets on Arc Testnet and the source chain, then fund them for the bridge flow. If you already have funded wallets on Arc Testnet and your source chain, skip to Step 3.

2.1. Create wallets

Install the Circle Dev-Controlled Wallets SDK:

```
npm install @circle-fin/developer-controlled-wallets
```

Then create wallets on Arc Testnet and your source chain, the example shows how to create wallets on Arc Testnet and Solana Devnet:

```
import { initiateDeveloperControlledWalletsClient } from "@circle-fin/developer-controlled-wallets";

const client = initiateDeveloperControlledWalletsClient({
  apiKey: "<YOUR_API_KEY>",
  entitySecret: "<YOUR_ENTITY_SECRET>",
});

const walletSetResponse = await client.createWalletSet({
  name: "Bridge Wallets",
});

/*
 * Testnet blockchains:
 * ETH-SEPOLIA | AVAX-FUJI | MATIC-AMOY | SOL-DEVNET | ARB-SEPOLIA
 * UNI-SEPOLIA | BASE-SEPOLIA | OP-SEPOLIA | ARC-TESTNET | MONAD-TESTNET
 */
const walletsResponse = await client.createWallets({
  blockchains: ["ARC-TESTNET", "SOL-DEVNET"],
  count: 1,
  walletSetId: walletSetResponse.data?.walletSet?.id ?? "",
});
```

> If you're calling the API directly, you'll need to make two requests: one to create the wallet set and one to create the wallet. Be sure to replace the Entity Secret ciphertext and the idempotency key in your request. If you're using the SDKs, this is handled automatically for you.

2.2. Fund your wallets

Obtain testnet USDC from the Circle Faucet and native tokens from the Console Faucet to pay gas fees on both the source chain and Arc Testnet.

2.3. Check wallet balances

You can check wallet balances from the Developer Console or programmatically:

Node.js cURL

```
const response = await client.getWalletTokenBalance({
  id: "<WALLET_ID>",
});
```

Step 3. Bridge USDC

In this step, you set up your script, execute the bridge transfer, and check the result.

3.1. Create the script

Create an `index.ts` file in the project directory and add the following code. Pick the source chain that matches the wallet you funded. Each example uses the exact Bridge Kit chain identifier accepted by the SDK.

Arbitrum Avalanche Base Ethereum Optimism Polygon Solana Unichain

```
// Import Bridge Kit and the Circle Wallets adapter
import { BridgeKit } from "@circle-fin/bridge-kit";
import { createCircleWalletsAdapter } from "@circle-fin/adapter-circle-wallets
import { inspect } from "util";

// Initialize the SDK
const kit = new BridgeKit();

const bridgeUSDC = async () => {
  try {
    // Set up the Circle Wallets adapter instance, works for both ecosystems
    const adapter = createCircleWalletsAdapter({
      apiKey: process.env.CIRCLE_API_KEY!,
      entitySecret: process.env.CIRCLE_ENTITY_SECRET!,
    });

    console.log("---------------Starting Bridging---------------");

    // Use the same adapter for the source and destination blockchains
    const result = await kit.bridge({
      from: {
        adapter,
        chain: "Arbitrum_Sepolia",
        address: process.env.EVM_WALLET_ADDRESS!, // EVM address (developer-co
      },
      to: {
        adapter,
        chain: "Arc_Testnet",
        address: process.env.EVM_WALLET_ADDRESS!, // EVM address (developer-co
      },
      amount: "1.00",
    });

    console.log("RESULT", inspect(result, false, null, true));
  } catch (err) {
    console.log("ERROR", inspect(err, false, null, true));
  }
};

void bridgeUSDC();
```

3.2. Run the script

Save the `index.ts` file and run the script in your terminal:

```
npm run start
```

> For blockchains other than Arc, you will need native tokens to pay for gas. The **approve** and **burn** steps require gas fees in the native token of the source chain, while the **mint** step requires gas fees in the native token of the destination chain.

3.3. Verify the transfer

After the script finishes, locate the returned `steps` array in the terminal output. Each transaction step includes an `explorerUrl` field. Use that URL to verify that the USDC amount matches the amount you transferred.

The following example shows how all four steps (Approve, Burn, Fetch Attestation, Mint) might look like in the terminal output. The values shown are for demonstration purposes only and don't represent a real transaction:

Approve Burn Fetch Attestation Mint

```
{
  name: 'approve',
  state: 'success',
  txHash: '0x809cd6678785c3cb48d73a8e35e5ef8fc21c1b1e22df767860bc30bd882fb470'
  data: {
    txHash: '0x809cd6678785c3cb48d73a8e35e5ef8fc21c1b1e22df767860bc30bd882fb47
    status: 'success',
    cumulativeGasUsed: 429183n,
    gasUsed: 38596n,
    blockNumber: 20523074n,
    blockHash: '0x5d81b5abab77f19fb3e70df620276fe9b4be68172482afa0188296ead97c
    transactionIndex: 5,
    effectiveGasPrice: 162000000000n
  },
  explorerUrl: 'https://testnet.arcscan.app/tx/0x809cd6678785c3cb48d73a8e35e5e
}
```

Summary

After completing this tutorial, you've successfully:

- Set up a Bridge Kit workflow for Circle Wallets, Viem, or Solana adapters
- Funded the source and destination wallets needed for the bridge flow
- Bridged USDC to Arc Testnet

Was this page helpful? Yes No

Deploy contracts Transfer USDC or EURC

Links

Network status

Blog

Community

Arc homepage

Resources

Terms of service

Privacy policy

Cookie policy

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.